How LibRate works

Keys, frames, hops and relays — explained without jargon.

Document 2 of 4 · Version 1.0 · Contact: keshavgithub01@gmail.com

1. The big picture



Two kinds of pipe carry the same messages. **Bluetooth** joins phones that are physically close. **Relays** join phones that have internet. Any phone that has both acts as a bridge. Nothing is configured; every phone simply does the same three things: listen, deliver, repeat once.

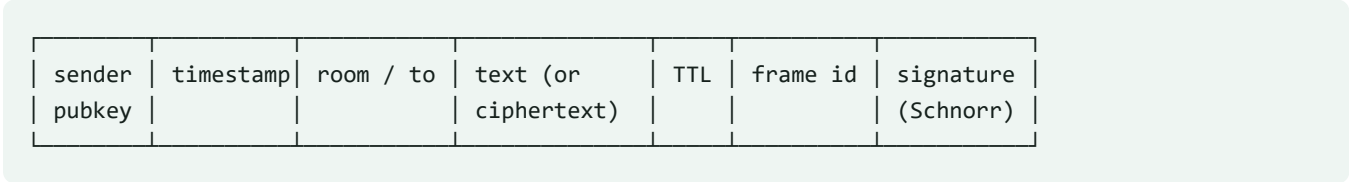
2. Your identity is a key

The first time LibRate opens it generates a **secp256k1 key pair** — the same type of key used by Nostr and Bitcoin. The *private* half is stored in the phone's secure storage and never leaves it. The *public* half is your address. From it the app derives a readable name (*Brave Otter*), an avatar and a colour, so people can recognise you without you typing anything. You can rename yourself at any time; the key underneath stays the same, which is why friends still recognise you.

Your public key is shown as an `npub…` string and as a QR code. Scanning it is how someone starts a private chat with you — there is no directory to search and no server to ask.

3. Every message is a signed frame

When you send text, the app packs it into a compact binary **frame**:



- The **signature** is made with your private key over everything except the TTL. Anyone can check it with your public key; nobody can forge or alter the frame.
- The **TTL** (time-to-live) starts at 7. Each phone that forwards the frame lowers it by one and drops it at zero. Because the TTL is outside the signature, forwarding does not break it.
- The **frame id** is a hash of the signed content. It is the same no matter which path the frame takes — this is what lets Bluetooth and relay copies collapse into one.

4. The Bluetooth mesh

Every phone is simultaneously a Bluetooth **advertiser** ("I am here") and a **scanner** ("who is here?"). When two phones find each other they open a link and exchange frames in both directions. Larger frames are split into pieces that fit the link and reassembled on the other side.

The forwarding rule is deliberately simple:

1. A frame arrives. Check the signature; if bad, drop it.
2. Have I seen this frame id before? If yes, drop it (duplicate).
3. Remember the id. Show the message if it is for a room I am in, or for me.
4. If TTL > 1: lower it by one and send the frame **once** to every other linked phone.

Step 2 prevents loops and storms; step 4 makes the message ripple outward. One hop reaches everyone within ~10–30 m, so the area covered grows with the square of the hop count: seven hops is roughly a 250 m square — tens of thousands of people at protest density, provided the crowd is contiguous and enough phones run the app.

> **Store & forward.** Phones hold recent frames for a while. When a new phone appears, anything it missed is handed over. A person walking from the front of a march to the back physically carries the messages with them — and they re-ripple on arrival.

5. Relays

A relay is a tiny public server speaking the open **Nostr** protocol: clients connect over a secure WebSocket (`wss://`), publish signed events, and subscribe to events they care about. LibRate wraps each frame in a Nostr event and uses the relay as a second pipe:

- A phone with internet **publishes** every frame it hears (from Bluetooth or from the user) to its relays.
- It also **subscribes**, and **injects** every frame that arrives from a relay into its Bluetooth neighbourhood.
- Because the frame id is identical on both pipes, duplicates are discarded exactly as in the mesh.

The result: one phone with one bar of signal connects a whole offline crowd to every other crowd using the same relays — in the next street or the next city. LibRate ships with a built-in relay list. Relays can also arrive inside QR codes: a room invite or someone's profile carries the relays they use, so groups naturally converge on the same servers. Anyone can run a relay in about ten minutes (see `docs/RELAY.md`), and relays can mirror each other so a blocked one is simply replaced.

6. Encryption

Nearby, open rooms	**Signed, not encrypted.** They are public by nature — like a megaphone. Encrypting them would add nothing.
Password rooms	The password is stretched with **Argon2id** into a key; each message is sealed with **XChaCha20-Poly1305**. The password itself never travels. Without it, relays and bystanders see noise.
Private chats	**NIP-44**: the two keys agree a shared secret (ECDH on secp256k1); messages are encrypted with ChaCha20 and authenticated with HMAC-SHA256, padded so lengths leak less. Only the two phones can read them.
Groups with admins	The creator publishes a **signed charter** (admins, description). Removals and dismantling are signed records too. Every phone verifies them independently — governance without a server.
On the phone	Messages, contacts and settings are stored encrypted with a device key. Screenshots are blocked by default. Cloud backup is disabled.

7. Delivery ticks and read receipts

Your own bubbles show a small status: *pending* (not yet handed to any pipe), *sent* (a relay acknowledged it, or a Bluetooth peer accepted it), *delivered* (a peer echoed or forwarded it), and in private chats *seen* (the other phone sent back an encrypted acknowledgement). Read receipts are themselves private messages, so relays cannot tell that one was sent.

8. Staying alive in the background

Meshing only helps if the app keeps running while the screen is off. LibRate runs a foreground service (the persistent notification you see) and keeps its engine alive across screen locks and app switches. Phone makers' battery managers (ColorOS, MIUI, One UI) are aggressive; the app asks once for an exemption and reminds you later if it was revoked. Without it, messages wait until you reopen the app.

9. Updates and data

Installing a newer APK over the existing one keeps everything — identity, contacts, rooms, messages. Android only allows this when the new file is signed with the **same key**, so a tampered "update" cannot replace a genuine install. Uninstalling deletes everything permanently; there is no copy anywhere else, by design.

10. Contact

Questions, bug reports, relay addresses, or a fresh download link if yours is blocked:

keshavgithub01@gmail.com

Always verify what you receive: compare the APK's SHA-256 with `SHA256SUMS.txt` and check the signing certificate `7F:05:8A:F1:48:8C:A4:0A:AF:B2:14:86:D7:7A:64:24:19:9B:00:30:BB:38:A0:44:FA:11:B7:8D:7A:78:15:B6`.
